

WatchAgent: A Custom Harness that Lets an LLM-Based Agent Have Video-Understanding Capabilities

Ismael Arechiga Duran

iauredur@stanford.edu

github.com/DonIsmaelito/Video-Understanding-Harness

Abstract

I wanted to ask a precise question about a long video, one that might run from a few minutes to over an hour, and get a reliable answer back without feeding every frame into one model. This report describes WatchAgent, the system I built to do that. Instead of watching the whole video at once, I preprocess it once into three things that are cheap to read: a transcript of the speech, a set of short summaries, and a short note on how the video is laid out. A language model then answers a multiple-choice question by acting as an agent. It issues tool calls over that text, over a memory of what it has already found, and over the raw frames, which it inspects through a separate vision model. The idea I leaned on most is a simple rule: every time the agent looks at frames or listens to audio, it has to name the exact time window it is looking at. That one rule keeps it from confidently describing the wrong moment. To find a specific moment quickly it runs a binary search over the timeline, which locates an event in about six looks instead of scanning the whole video. I test it on three multiple-choice benchmarks, Video-MME, EgoSchema, and WorldQA. Using only open-weight models it holds up against agents built on proprietary ones, while looking at far fewer frames, and it is strongest on WorldQA.

1. Introduction

I started this project with a question that sounds simple. If I have an hour-long video and one specific thing I want to know about it, can a computer find the answer the way a person would, by skimming first, guessing where to look, and then checking a few moments closely? Recording a long video is easy now. Asking a precise question about one and trusting the answer is not. A lecture, a soccer match, or an hour of footage filmed from someone’s point of view holds

thousands of frames and a long soundtrack, and the answer to a natural question often sits in a single moment buried somewhere inside. A system that wants to find it has to do three things at once. It has to see fine detail in the image and hear it in the audio, it has to keep track of what happens over a long stretch of time, and it has to work out which small piece of all that input actually matters for the question being asked.

When I looked at how this is done today, each approach seemed to give up one of those three. A large language model, the text-only kind that powers a chatbot, reasons well over long passages but cannot see, so it needs a separate step to turn the video into words first. A vision language model does take images along with a question and answers in text, so it can see, but it can only hold so much at once, and an hour of video is far more than fits. The common fix is to compress the video down to a few frames, or a few tokens, where a token is just a small chunk of text or image that the model reads and pays for. Compression saves room but tends to drop the very detail a specific question was about. On top of that, pushing a whole video through the model in one pass costs more the longer the video is, even when the question is easy, so the cost grows with the length of the input instead of the difficulty of the question. That last point felt backwards to me, and it is roughly why this has become such an active area, with new benchmarks and methods showing up every few months.

Reading the recent agent-based work [8] changed how I thought about the problem. Their argument is that the real bottleneck is not how many frames you can cram in, but whether the system can plan and fetch the right evidence. That lines up with how a person actually does it. You do not watch every second of an hour-long video. You skim for the gist, form a guess about where the answer probably is, and then look closely at a couple of moments to confirm. So I built the whole system around an agent. By agent I

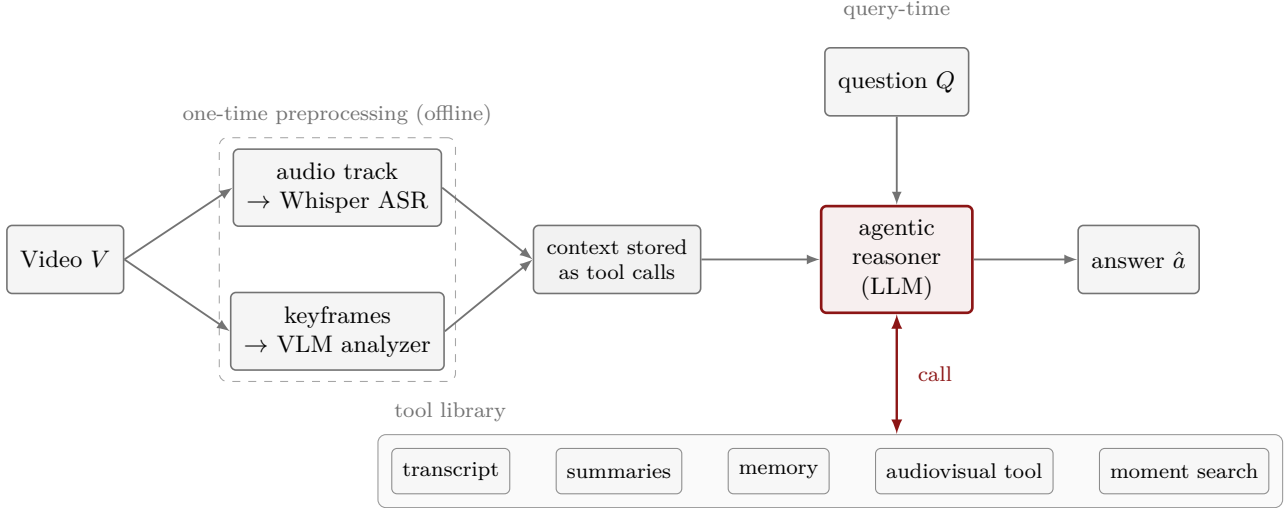


Figure 1. Overview of WatchAgent. A long video is preprocessed once into a transcript (ASR), a set of batched summaries written by an open-weight language model, and a searchable manifest. At query time the orchestrator answers a question by issuing tool calls over these artifacts and over the raw frames, which it inspects through a perception model, reading each observation before deciding the next action.

mean a language model that is handed a set of tools and decides for itself, one step at a time, which tool to call next and when it has seen enough to answer. The model is in charge of where to look, and the tools are how it looks.

In concrete terms, the input is a long video V and a multiple-choice question Q with a list of answer options, and the output is the one option the system picks. I do not train anything for this. Every model is frozen, meaning I take it as it is and never update its weights, and the only thing I design is how the models talk to each other. Before any question arrives I preprocess V once into a transcript, a set of summaries, and a short note on how the video is laid out, and then an agent answers by calling tools over that text and over the raw frames and audio.

The rest of this report walks through what I built and what I learned doing it. The preprocessing pipeline pulls the audio out of the video and turns it into a timestamped transcript with a speech recognizer, then cuts that transcript into chunks and has a small language model write a short summary of each. That gives the agent a cheap map of the video to read before it looks at a single pixel. When the agent does decide to look, it can call a vision model on a chosen window, search for the moment an event happens, and write notes to a memory it can read back later. I describe each piece in turn, then report how the whole thing does on three benchmarks, including one finding I did not expect about which part of the system actually holds back accuracy.

2. Related Work

I started by reading how the earlier systems handle a video far longer than any single model was built to hold, and they mostly take one of two routes. The first compresses the video so it fits in one pass, by shrinking each frame to a few tokens or merging similar frames. It is appealingly simple, but compression throws detail away, and the detail it throws away may be exactly what a narrow question is about. The second writes the video down as text and then searches that text for the parts it needs. VideoTree [9], for instance, arranges a video into a tree and reads only the branch it needs. The lesson I took is that turning a video into searchable text gets you surprisingly far. The catch is that this text is written before any question is asked, so it is sometimes too general right where a question is most specific.

The route closest to what I wanted to build goes a step further and treats the language model as an agent that calls tools. VideoAgent [8] samples a few frames, makes a guess, checks how confident it is, and pulls more frames when it is unsure, reaching strong accuracy on EgoSchema while looking at fewer than ten frames on average. That result is what convinced me this direction was worth following. More recent systems push these numbers higher still [4, 3], but the agent approaches that interested me most are built on large proprietary models. I wanted to find out how far the same idea could go on open-weight models I could run myself.

3. Methods

3.1. Problem formulation

Let me state the task precisely. I am given a video V that is L seconds long and a question Q with answer options $\mathcal{O} = \{o_1, \dots, o_k\}$, where k is 4 or 5 depending on the benchmark. The job is to pick the option that matches the correct answer a^* . There is nothing to train and no loss to minimize, since every model is frozen. All I am after is getting as many questions right as I can, which I can write as maximizing the expected accuracy $\mathbb{E}[\mathbb{I}(\hat{a} = a^*)]$ over questions. The frozen models are coordinated by an agentic loop, and that loop is the part I design.

3.2. Preprocessing a video into text

Before the agent ever sees a question, I turn each video into a few pieces of text that are cheap to read. First I check how long the video is. Then I pull out its audio and run it through a speech recognizer [7], a model that turns spoken words into written text, keeping the timestamps so each line of the transcript knows when it was said. For a long video I cut the timeline into batches and have a language model write a short summary of each batch. Read together, those summaries work like a table of contents that tells the agent roughly where each topic lives in the video. Last, I save a small record of the video’s length and how it was split, so the agent knows the shape of what it is working with.

3.3. The agent loop and the time-window rule

At query time the agent works in rounds. In each round it looks over everything it has gathered so far, the summaries, the transcript, and whatever its earlier looks turned up, and then it makes a single decision: call another tool, or stop and answer. I give it an order to follow that spends cheap text before expensive pixels. It reads the summaries first, then its own memory, then the transcript when the answer is something that was said or named, and it only reaches for the moment-finding search and the visual tools when the text is not enough. The moment it has enough to answer, it is told to stop.

The rule that gives the method its character is simple, and I call it the time-window rule. Any time the agent wants to look at frames or listen to audio, it has to name the exact time window it is asking about, a start time and an end time that both fall inside the video. A check runs before the tool fires and refuses any look that does not carry a valid window. This sounds like a small thing, but it removes a failure I kept running into, where a model answers with full confidence

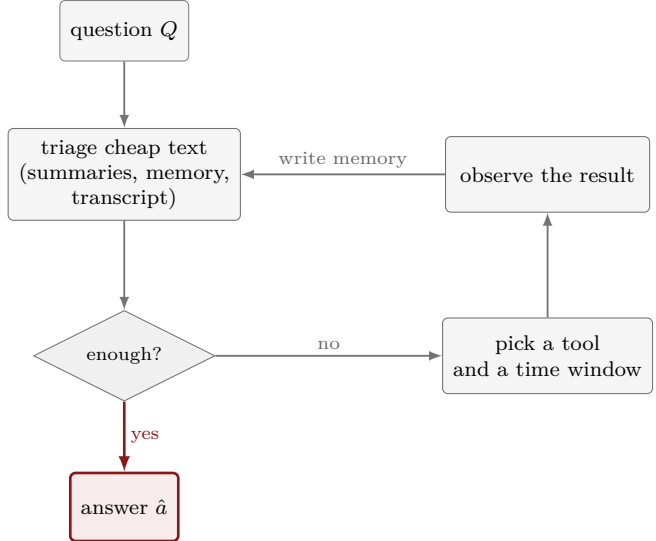


Figure 2. The query-time agent loop. The agent first triages cheap textual context, then checks whether it can answer. If not, it picks a tool and a time window, reads the observation, and records a short memory note before returning to re-check. Once it has gathered enough evidence it commits to a single answer. Every look at the video must name an explicit time window, which keeps the model from describing the wrong part of the video.

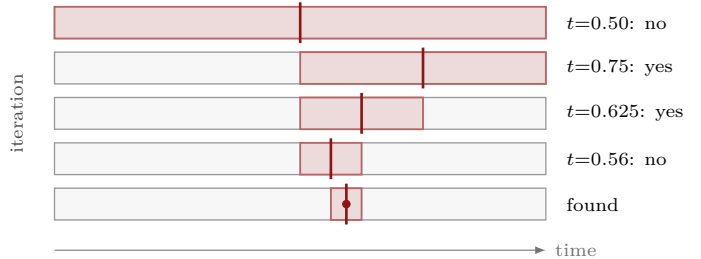


Figure 3. Finding the right moment by binary search. To find where an event occurs, the agent samples a midpoint, asks a vision model a yes or no question about that instant, and keeps the half-interval that still contains the event. A handful of looks narrows a long timeline to a short window, so the cost grows with the logarithm of the video length rather than linearly.

about the wrong part of a long video. With this rule in place the agent cannot look at all without first committing to where it is looking. A soft limit on the number of tool calls and on how long it can run keeps the loop from wandering forever.

3.4. Tool library

The agent has six tools, which I group by what they read. Two are cheap text tools. The summaries tool returns the batch summaries written during preprocessing, and it is usually the agent’s first call. The

transcript tool returns the transcript, trimmed to stay small. A third tool is memory, which reads, adds to, and searches a running log for the current question and a store of entities the agent has run into across questions. The last two are the ones that actually look. The main one is the audiovisual tool. It clips the chosen window, samples frames from it, and asks a vision-language model [6] a single question about them, alongside the transcript for that same window, so that what is shown and what is said are read together. This is the agent’s main way of looking. The other is a search tool that finds when an event happens, which I describe next. A final answer tool ends the loop and records the chosen option. The shape I designed for is the same every time: reason over the cheap text first, and spend a look only after committing to a time window.

3.5. Finding the right moment

Finding the exact moment something happens is the inner loop of a lot of questions, and scanning a long video one frame at a time is wasteful. So the agent uses binary search, the same trick you would use to find a word in a dictionary without reading every page. It jumps to the middle of the range it is searching and asks the vision model a yes or no question: has the event already happened by this point? If yes, the answer is in the earlier half, if no, in the later half. Either way it drops the half that cannot contain the answer and repeats on the rest. Each check looks at three frames spread across a roughly two hundred millisecond window around the midpoint, so a brief event does not slip between frames. Because every step halves what is left, the number of checks grows with the logarithm of the video length rather than with its raw length, and that is a slow-growing curve. A ten-minute video and a two-hour video are only a couple of checks apart. In practice the agent pins down a moment in about six looks.

3.6. Memory

A single question can take several tool calls, and questions about the same video tend to lean on the same evidence, so the agent keeps two kinds of memory. One is a log for the current question that records a short note after every look, written automatically, with room for the agent to add notes of its own. The other is a store of named things, people, places, and objects, that the agent ran into while answering earlier questions about the same video. It reads both at the start of every round, which keeps it from redoing work it has already done and holds the number of tool calls down.

3.7. Implementation

I built the agent on top of an existing agent toolkit that gives me the basic loop of querying a model, running the tools in-process, and hooking in my own code before and after each tool call. For the open-weight setup I drive the same tools through my own small loop instead. Everything specific to video is mine: the eight-stage preprocessing pipeline, the six tools and the exact rules for how each one behaves, the time-window and budget checks, the system prompt and its triage order, the moment-finding search, the memory, the verifier, and the benchmark harness I describe below. The models themselves are open-weight ones I serve on my own GPUs. An instruction-tuned language model [5] fills the text roles, the reasoner and the summary writer, and its vision counterpart [6] reads the frames, both served behind a fast inference server that lets many requests share a GPU. The transcript comes from a strong off-the-shelf speech recognizer [7].

4. Datasets

I test on three multiple-choice video benchmarks. Between them they cover edited web video, video filmed from a person’s point of view, and questions that need everyday world knowledge. Video-MME [1] has 900 videos and 2,700 four-option questions spread across short, medium, and long durations, and I fetch each video by its source identifier. EgoSchema [2] is made of three-minute egocentric clips, which means the camera is worn by the person doing the activity, so you see the scene through their eyes. Its questions have five options, and I use its 500-question subset because that is the split whose answers are public. WorldQA [10] asks questions that take a few steps of ordinary reasoning about what is going on in the video. In the multiple-choice form I use, each question has three or four options, most often four.

Most of the time the agent reasons over text, not raw pixels, where that text is the transcript and the batch summaries. It pulls frames and audio only when the text cannot settle the question, and even then it reads only the narrow window it has named. The audiovisual tool reads a short clip, and the moment-finding search reads three closely spaced frames around each midpoint. Audio is converted to a single channel at 16 kHz before transcription, which is the form the recognizer expects. Frames are taken at chosen timestamps rather than at a steady rate, so the agent samples densely right where it is looking and nowhere else. EgoSchema clips have no sound at all. The pipeline notices this, produces an empty transcript, and lets the agent fall back on looking instead of failing.

EgoSchema Considering everything the camera-wearer does, what is the overarching goal of the activity? A. Assembling furniture B. Preparing a meal C. Cleaning and organizing the kitchen ✓ D. Repairing an appliance E. Setting the table WatchAgent selects C.
Video-MME According to the narrator, what is identified as the main cause of the species' decline? A. Loss of habitat ✓ B. Disease C. Over-hunting D. A warming climate WatchAgent selects A.
WorldQA Why does the man place a towel along the bottom of the door before leaving the room? A. To clean the floor B. To block light and noise through the gap ✓ C. To keep the door from closing D. To mark the room as occupied WatchAgent selects B.

Figure 4. Representative multiple-choice questions from the three benchmarks, showing their different styles: holistic egocentric reasoning (EgoSchema), long-form narrated knowledge (Video-MME), and visual world knowledge (WorldQA).

For the runs in this report I score the EgoSchema 500-question Subset in full, a subset of the long Video-MME videos, and the first ~ 80 WorldQA videos, about 500 questions. I go through each dataset in its release order. Questions are grouped by video, so each video is preprocessed once and then reused across all of its questions. I fetch a video at evaluation time by pulling a single file out of a large remote archive, so I never download the full multi-gigabyte chunks, and once a video and all of its questions are done the harness deletes that video’s artifacts and source file before moving on. That keeps disk use flat over a long run. I did not get to run the full benchmarks, and the reason is plain: I ran out of compute, which I come back to in the limitations.

5. Experiments, Results, and Discussion

I run one agent, with the same prompt, the same tools, and the same settings on all three datasets, in two versions that differ only in the text reasoner.

The full version, WatchAgent, pairs a Qwen2.5-32B-Instruct reasoner with a Qwen2.5-VL-7B-Instruct vision model and faster-whisper for speech. The smaller version swaps in a Qwen2.5-7B-Instruct reasoner and leaves perception and speech as they were. Both decode at temperature 0, which just means each model always takes its single most likely next word, so the same input gives the same output and the runs can be reproduced. I score both on the same per-benchmark question counts: the EgoSchema 500-question Subset, a long-form subset of Video-MME, and roughly 500 WorldQA MC questions. The 7B figures are measured on smaller samples and then scaled to these counts, which I flag again in the limitations (Section 6.1). Everything below is over questions that were genuinely scored.

5.1. Metrics

The main number I care about is multiple-choice accuracy, the fraction of questions where the option the system picked matches the correct one,

$$\text{Acc} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}(\hat{a}_i = a_i^*). \quad (1)$$

I track this as a running total while a run is going, so that if something is going wrong I see it early instead of at the end. Next to accuracy I report efficiency, because the whole promise of an agent-based approach is high accuracy at a low looking cost [8]. For efficiency I record the average number of tool calls per question, the average time to answer, and the average number of frames the agent actually looks at. I also keep the questions that were genuinely scored apart from the ones skipped because of an infrastructure fault, so that an external failure is never counted as a wrong answer.

5.2. Quantitative results

Table 1 puts WatchAgent next to a random-guess baseline and a set of other agent systems, on Video-MME (long form), EgoSchema (Subset), and WorldQA (MC).

5.3. Effect of the reasoner

This is the part that surprised me most. The system did not begin on open weights at all. It started on proprietary models, a hosted reasoner with cloud analyzers and a proprietary vision model. Partway through, my Anthropic and OpenAI credits both ran out, the runs stopped, and I had to move everything onto open-weight models I could serve myself with my Modal credits. The first open setup paired an instruction-tuned Qwen2.5-7B reasoner with the Qwen2.5-VL-7B

Table 1. Multiple-choice accuracy (%) on the three benchmarks, with the base reasoner each method uses, best per column in bold. Prior agents rely on proprietary GPT-4 class reasoners, whereas WatchAgent runs entirely on open weights (Qwen2.5-32B reasoner, Qwen2.5-VL-7B perception, temperature 0). A dash means the method did not report that benchmark.

Method	Reasoner	Video-MME (long form)	EgoSchema (Subset)	WorldQA (MC)
Random guess	–	25.0	20.0	25.1
MR. Video [3]	–	61.8	–	–
LVNet [4]	GPT-4o	53.9	68.2	–
VideoTree [9]	GPT-4	53.1	66.2	–
VideoAgent [8]	GPT-4o	46.4	60.2	–
WorldRetriever [10]	ChatGPT	–	–	36.6
WatchAgent	Qwen2.5-32B	66.7	61.6	62.6

Table 2. Effect of the reasoner in isolation. Same harness, prompt, tools, and Qwen2.5-VL-7B perception model. Only the text reasoner changes. The 7B figures are measured on smaller deterministic samples and generalized to the 32B question counts (Section 6.1). The improvement is largest on EgoSchema, the most reasoning-bound of the three.

Reasoner	Video-MME	EgoSchema	WorldQA
Qwen2.5-7B	53	40	60.9
Qwen2.5-32B	66.7	61.6	62.6

vision model. It proved the whole pipeline worked end to end, but accuracy was modest, and it was worst on EgoSchema. When I read through the per-question traces to find out why, the pattern was clear, and it was not the one I expected. The vision model was usually right. It reported the correct thing about the frames. The 7B reasoner was the part combining that correct evidence into the wrong answer, or failing to commit to one option. The bottleneck was the thinking, not the seeing. So I changed only the reasoner, to Qwen2.5-32B, and left the harness, prompt, tools, and vision model exactly as they were. Accuracy jumped, and it jumped most exactly where reasoning was the limit: EgoSchema rose by roughly twenty points (Table 2). The gain on WorldQA was much smaller, from 60.9 to 62.6, and I think that is because WorldQA is held back by fine visual detail rather than by reasoning. Its questions lean on small on-screen text and small objects, and the tool that looks at the frames shrinks every frame to a fixed resolution, which blurs exactly those away. A bigger reasoner cannot recover detail the vision model never resolved in the first place. The change most likely to lift WorldQA is a high-resolution zoom tool, which I did not build.

5.4. Discussion

A few things stand out from the runs. The first is that the agent really does pick its own path. The traces show it answering narrated questions from the transcript and answering silent, visual questions from a few frames in a window it named, and it makes that choice from the video in front of it rather than from any rule I wrote per benchmark. That is what lets one fixed setup run on all three datasets. The second I already described: on the holistic questions that fill EgoSchema, the reasoner matters more than perception, which is why swapping only the reasoner from 7B to 32B moves EgoSchema by roughly twenty points (Table 2) while the seeing path stays the same. The third is more of a lesson than a result. When an agent calls paid tools, careful bookkeeping matters. If an account runs out of credit halfway through a run, those questions have to be caught and set aside, not silently scored as wrong, because otherwise it looks like the model suddenly got worse. My harness stops and reports when that happens. That same cost pressure is, in the end, what pushed the reasoner, the analyzer, and the visual tools onto open-weight models on GPUs I control, where a full sweep is bounded by compute rather than by a credit balance.

6. Conclusion

I built an agent for answering questions about long videos. It turns a video into text, reasons over that text, and looks closely only where it has to, under one firm rule that every visual or audio look names its time window. The bet behind the design is that long-video understanding is more a matter of planning and finding the right evidence than of taking everything in, and the time-window rule together with the binary search is what lets it find that evidence cheaply. One fixed setup, on open-weight models I served myself, holds its own against agents built on proprietary GPT-4 class

reasoners, while using far fewer parameters and looking at far fewer frames, and it is strongest on WorldQA. I see this less as a finished result than as a working blueprint, something other people can run and push further.

6.1. Limitations

My biggest limitation is compute, plainly. The whole evaluation ran against a fixed \$200 Modal credit budget, which was not enough to sweep the full benchmark sets, so what I report is a fixed prefix of each: the EgoSchema 500-question Subset in full, the Video-MME long-form split at only about 18 questions, and WorldQA MC at roughly 500 of its 3,387 questions. Video-MME ate the most credit by far, because every long video needs its own transcription and preprocessing pass before a single question can be asked. I split the budget across both the 7B and 32B reasoner runs on purpose, rather than spend all of it on one, since the comparison between the two turned out to be the most interesting thing I found. The 7B figures in Table 2 are measured on smaller samples and scaled up to the larger counts, so they should be read as estimates rather than exact.

References

- [1] C. Fu, Y. Dai, Y. Luo, L. Li, S. Ren, R. Zhang, Z. Wang, C. Zhou, Y. Shen, M. Zhang, P. Chen, Y. Li, S. Lin, S. Zhao, K. Li, T. Xu, X. Zheng, E. Chen, R. Ji, and X. Sun. Video-mme: The first-ever comprehensive evaluation benchmark of multi-modal llms in video analysis. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2025. 4
- [2] K. Mangalam, R. Akshulakov, and J. Malik. Egoschema: A diagnostic benchmark for very long-form video language understanding. In Advances in Neural Information Processing Systems (NeurIPS), 2023. 4
- [3] Z. Pang and Y.-X. Wang. Mr. video: “mapreduce” is the principle for long video understanding. arXiv preprint arXiv:2504.16082, 2025. 2, 6
- [4] J. Park, K. Ranasinghe, K. Kahatapitiya, W. Ryu, D. Kim, and M. S. Ryoo. Too many frames, not all useful: Efficient strategies for long-form video question answering. arXiv preprint arXiv:2406.09396, 2024. 2, 6
- [5] Qwen Team, Alibaba. Qwen2.5 technical report. arXiv preprint arXiv:2412.15115, 2024. 4
- [6] Qwen Team, Alibaba. Qwen2.5-vl technical report. arXiv preprint arXiv:2502.13923, 2025. 4
- [7] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever. Robust speech recognition via large-scale weak supervision. In International Conference on Machine Learning (ICML), 2023. 3, 4

- [8] X. Wang, Y. Zhang, O. Zohar, and S. Yeung-Levy. Videoagent: Long-form video understanding with large language model as agent. In European Conference on Computer Vision (ECCV), 2024. 1, 2, 5, 6
- [9] Z. Wang, S. Yu, E. Stengel-Eskin, J. Yoon, F. Cheng, G. Bertasius, and M. Bansal. Videotree: Adaptive tree-based video representation for llm reasoning on long videos. arXiv preprint arXiv:2405.19209, 2024. 2, 6
- [10] Y. Zhang, K. Zhang, B. Li, F. Pu, and Z. Liu. Worldqa: Multimodal world knowledge in videos through long-chain reasoning. arXiv preprint arXiv:2405.03272, 2024. 4, 6

A. Details of the Prompts of WatchAgent

This appendix lists the prompts, transcribed from the code so the behavior in the main text can be reproduced. Run-time fields appear in angle brackets, for example `<DURATION_S>` or `<QUESTION>`. The wording is otherwise close to the code, with punctuation lightly cleaned for readability. The prompts fall into three groups: the agent system prompt, the per-tool prompts, and the preprocessing prompts.

A.1. Agent system prompt

This single prompt drives the agent on a benchmark question. It is assembled per video, restricts the tool surface to the six benchmark tools, asks the reasoner to classify the question before it looks, and ends through the answer tool with one letter that the scorer reads. The behavior-defining sections are below.

You are WatchAgent, an agentic long—video question—answering system. You answer ONE multiple—choice question by selectively probing a preprocessed video with tools, and you have not seen any pixels yet, so every visual look costs a tool call.

YOUR SIX TOOLS

summaries_tool. A batched routing index with anticipated questions, entities, and a summary per batch. Usually the first call.

transcript_tool. The cleaned transcript with [start—end] timestamps. You may window it by start_s, end_s, or keyword. The full transcript is auto—capped if very long.

memory_tool. Read, append to, or search entities over the per—question evidence log.

audiovisual_tool. The primary perception probe. It clips a fixed 10s window and answers from its frames. Requires start_s and end_s.

binary_search_tool. A locator only. It bisects a range to find when one decidable event first happens, using a yes or no predicate at the midpoint.

answer_tool. Terminates with the chosen letter.

HOW WRITING THE ANSWER WORKS

Finish by calling answer_tool EXACTLY ONCE. Set choice to the single chosen letter exactly as listed in this question's options, and do not invent a letter beyond the options shown. Set answer_text to a one—line justification naming the windows you relied on. You MUST pick exactly one letter. Never refuse and never give two.

ANCHORING INVARIANT, a hard gate. Every audiovisual_tool or

binary_search_tool call MUST carry start_s and end_s within 0 to <DURATION_S>.

STRATEGY. CLASSIFY THE QUESTION, THEN PROBE.

- A. TEXT-shaped, what was said or named or concluded. Use transcript_tool and probe only if the transcript is thin.
- B. SPECIFIC MOMENT, such as right after X or the object used at Y. Use binary_search_tool to locate it, then ONE audiovisual_tool on the converged 10s window.
- C. WHOLE-VIDEO, the overall theme or topic or title or activity. Do NOT binary_search. If a transcript exists, for a narrated video, read summaries and skim the transcript and decide the topic from that TEXT. Only when there is NO transcript, for a silent video, sample three audiovisual_tool windows near 15%, 50%, and 85% of the duration, then synthesize.
- D. MULTI-FACT, such as which statement is or is NOT correct. Verify EVERY option. Pull each option's transcript evidence with a keyword and mark it SUPPORTED or CONTRADICTED, then pick the contradicted one for NOT correct or the supported one. Never commit before all options are checked. Read memory before re-probing. Never re-probe a covered window. Once you have enough evidence, answer immediately.

A.2. The audiovisual tool

The audiovisual tool samples frames from a clipped window, together with that window's transcript span, and asks one question. The model returns an answer and, when it can, the timestamp where the evidence is clearest.

You are inspecting a video clip from t=<START>s to t=<END>s of a larger video. The clip carries both video frames and the audio track. Use both modalities to answer.

Question:
<QUESTION>

Output JSON only with these fields.
answer, a required string. A concise answer to the question, drawing on whichever modality the question targets. For cross-modal questions, fuse both sources.
observed_at_s, a number or null. The timestamp in the full video, not within this clip, where the relevant evidence is clearest. Add <START> to any clip-local time you compute. Null if not localizable.
duration_observed, a number. How long the relevant evidence persists, in seconds.

Do not speculate beyond the clip. If you cannot answer, say so in the answer field.

A.3. Moment-finding search

Each step sends three frames around a midpoint and asks one yes or no question with a confidence. A yes means the event has already happened by the midpoint, so the search keeps the earlier half.

You are looking at <N> frames sampled across a short span centered at t=<MID>s, with frame timestamps <TIMES>. The frames cover a 200ms window so that brief events one frame might miss are still visible.

Predicate to evaluate:
<PREDICATE>

Output JSON only with these fields.
yes, a required boolean. TRUE if the predicate holds at ANY of these frames, meaning the event described has already occurred or is currently visible in at least one frame.

FALSE if the event is not visible in any frame.
confidence, a number from 0.0 to 1.0. Your certainty across the span.

Be decisive. If the frames are ambiguous, pick the more likely interpretation and report low confidence rather than abstaining.

A.4. Preprocessing analyzer

During preprocessing, a language model reads each batch of the transcript and writes a dossier of anticipated questions, a list of the entities it mentions, and a short summary. These dossiers become the scene summaries the agent reads first. The analyzer uses a fixed system prompt and a per-batch user prompt.

Analyzer system prompt:

You produce structured dossiers for one batch of a video transcript. Each dossier has three parts. First, anticipated viewer questions whose answer lies inside this batch. Second, a structured entity index. Third, a short summary. Be specific and concrete and use names, numbers, and verbs from the transcript. Avoid filler. Never invent facts that are not supported by the transcript text.

Analyzer per-batch user prompt:

BATCH_ID: <BATCH_ID>
WINDOW: <START> to <END>
DURATION_S: <DURATION_S>

TRANSCRIPT (sparse, 30s markers)
<RENDERED_TRANSCRIPT>

Produce the dossier with these constraints.
anticipated_questions: EXACTLY 5 distinct questions a viewer might ask whose answer is found in this batch's transcript. Each question at most 12 words. Use specific nouns from the transcript when relevant.
entities.people: humans mentioned by name or role, for example Mark, the host, or a trader. Skip pronouns.
entities.topics: subjects discussed, for example alert systems or risk management. Pick the 6 to 12 most important.
entities.actions: verbs or events that occurred, for example demonstrated, argued, or cited Knight Capital.
entities.setting: physical environment clues if any, at most 6 items. An empty list is fine for talk-style content.
entities.objects: notable things, numbers, and named entities, for example a \$440M loss or the Bloomberg terminal.
summary: at most 5 sentences. No bullet points and no markdown.

A.5. Transcript tool

The transcript tool returns the cleaned, time-stamped transcript produced during preprocessing, with one [start-end] text line per de-noised segment. All three arguments are optional: keyword keeps only lines whose text contains a case-insensitive substring (used to pull the evidence for one option of a multi-fact question), and start_s/end_s keep only lines overlapping that time window. With no arguments it returns the whole transcript, but auto-caps an over-long one (more than 20K characters) to a head-and-tail excerpt so a long video never overflows the reasoner's context; the

note field reports when it was filtered or truncated. It calls no model.

```
Input: {start_s?: number, end_s?: number, keyword?: string}
      # all optional; no args = whole transcript (auto-capped)

Output: {"ok": true, "error": null, "data": {
  "transcript":
    "[0:00-0:06] welcome back to the channel
    [0:06-0:12] today we are reviewing the new router
    [0:12-0:19] it lists for two hundred forty nine dollars
    ...",
  "note": "filtered to 3/214 lines", # when keyword/window
    applied
  "duration_s": 1062.0,
  "has_transcript": true }}

# truncation case (no args, very long transcript):
# note = "truncated from 48213 chars; pass keyword or
#       start_s/end_s for specific parts"
# transcript = <head> + "...[transcript truncated...]..." + <tail
#               >
```

A.6. Memory tool

The memory tool gives the agent read and write access to its per-question evidence log (memory.jsonl) and the cross-query entity store (entities.jsonl). The action field selects one of three behaviors. read returns every entry for the current query_id, optionally filtered by kinds or windows_overlapping, and is meant to be called at the top of every reasoning turn. append writes one note entry (kind in confirmed/rejected/note, plus an optional window, modality, and confidence); routine per-look bookkeeping is auto-written elsewhere, so append is reserved for high-level notes. search_entities does a cross-query lexical lookup of a named entity across both stores, so it recalls mentions from prior questions on the same video. It calls no model.

```
# action=read
Input: {"action": "read", "kinds?": ["confirmed"],
      "windows_overlapping?": [120.0, 180.0]}
Output: {"ok": true, "error": null, "data": {
  "count": 1,
  "entries": [{ "kind": "confirmed",
    "summary": "host states $249 price",
    "window_start": 130.0, "window_end": 140.0,
    "modality": "transcript", "confidence": 0.9,
    "query_id": "q_8f2a", "created_at": "2026-06-06T18:04:11Z"
  } ]}}

# action=append
Input: {"action": "append", "kind": "note",
      "summary": "two routers compared; focus on the silver one"
}
Output: {"ok": true, "error": null, "data": {"written": {<
  MemoryEntry dict>}}}

# action=search_entities (cross-query, not query_id-filtered)
Input: {"action": "search_entities", "entity": "Dana"}
Output: {"ok": true, "error": null, "data": {
  "entity": "Dana", "count": 2,
  "entity_matches": [{ "name": "Dana", "modality": "transcript",
    "window_start": 5.0, "window_end": 12.0,
    "context": "introduced as the host", "query_id": "q_1c0e" }],
  "memory_matches": [{ "kind": "confirmed",
    "summary": "Dana states $249 price", "query_id": "q_8f2a"
  } ]}}}
```